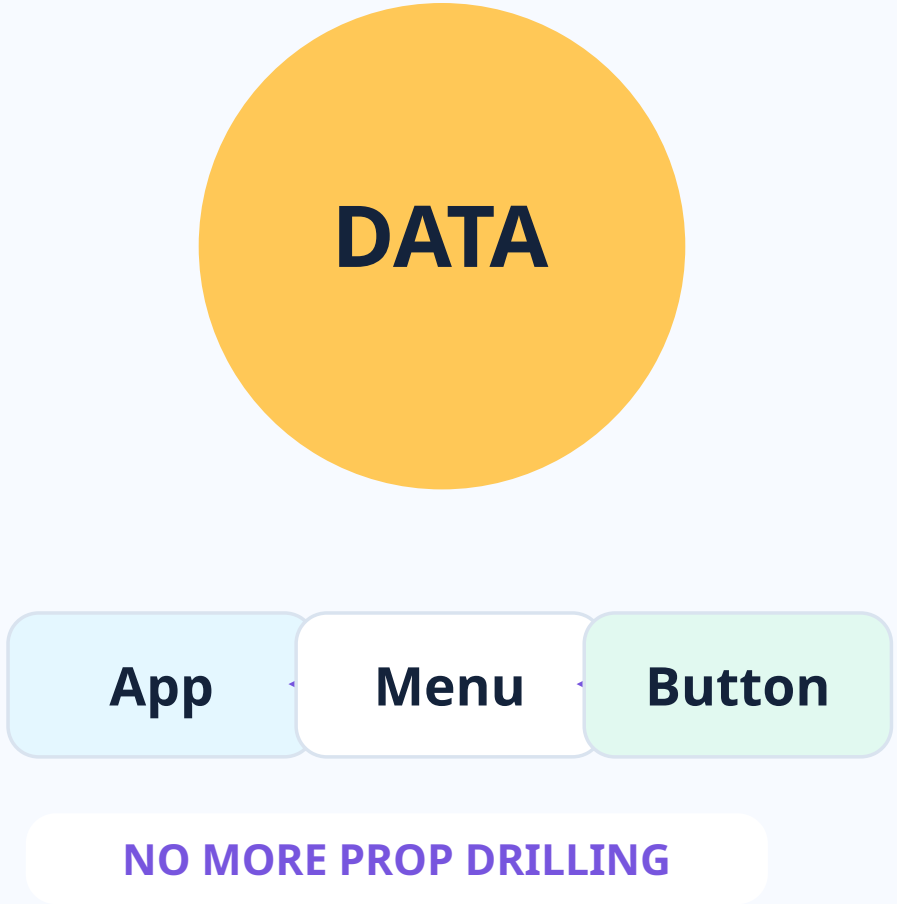
THE SECRET SHARING POWER

React Context API • A visual adventure for beginner developers

Send data to faraway components...

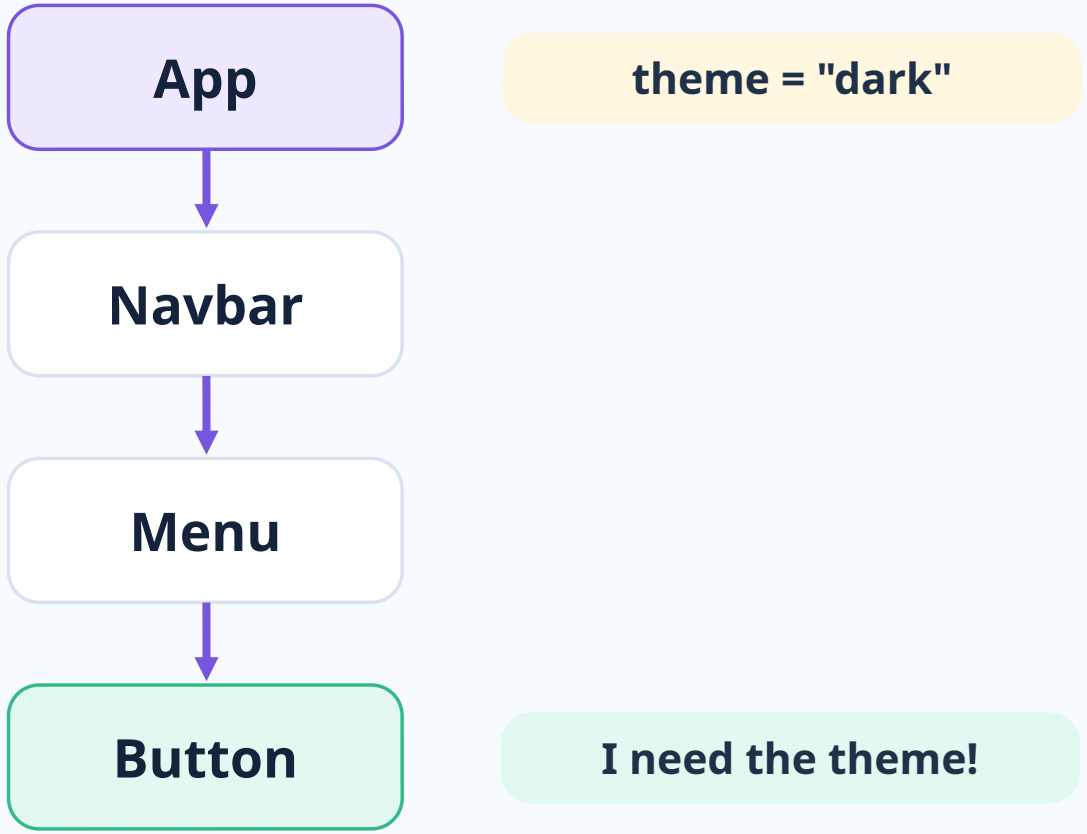
...without passing props through every level!



Meet our tiny app

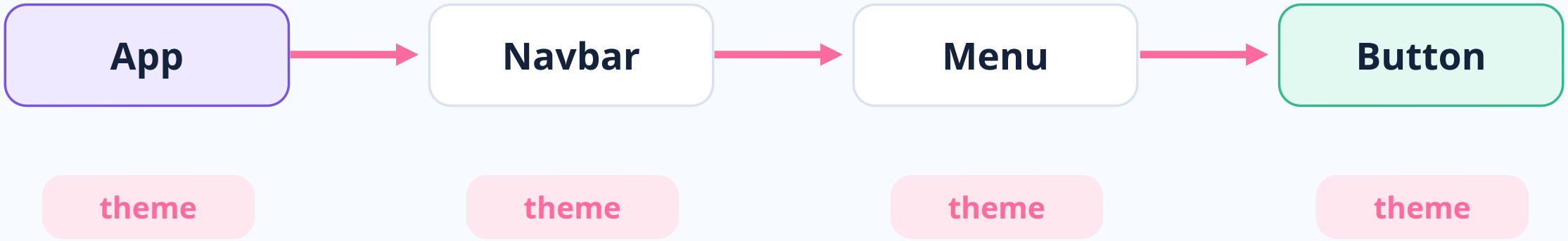
The theme color starts in App, but Button needs it.

The middle components don’t actually need it.



The problem: PROP DRILLING

Data is passed from person to person, even if most do not need it.

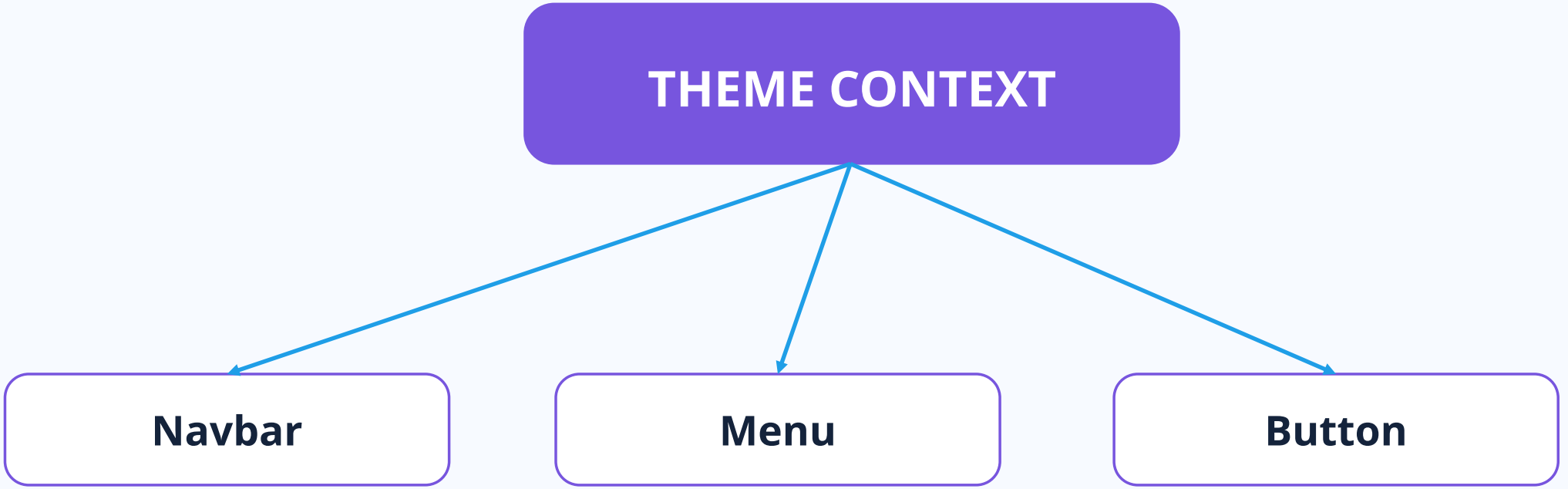


Too many props = repetitive code + harder refactoring

Props are NOT bad. Passing one or two levels is perfectly fine!

Context to the rescue!

Imagine a shared noticeboard that all descendants can read.



Reads when needed

The middle components do NOT have to forward theme props.

The 3-step recipe

CREATE → PROVIDE → READ

1

CREATE

Make a context

```
createContext()
```

2

PROVIDE

Put a value above the children

```
<ThemeContext value={...}>
```

3

READ

Get it inside any descendant

```
useContext(ThemeContext)
```

STEP 1 — Create your context

ThemeContext.jsx • Make one shared context object

NEW FILE

```
import { createContext } from 'react';

export const ThemeContext =
  createContext('light');
```

DEFAULT VALUE

"light"

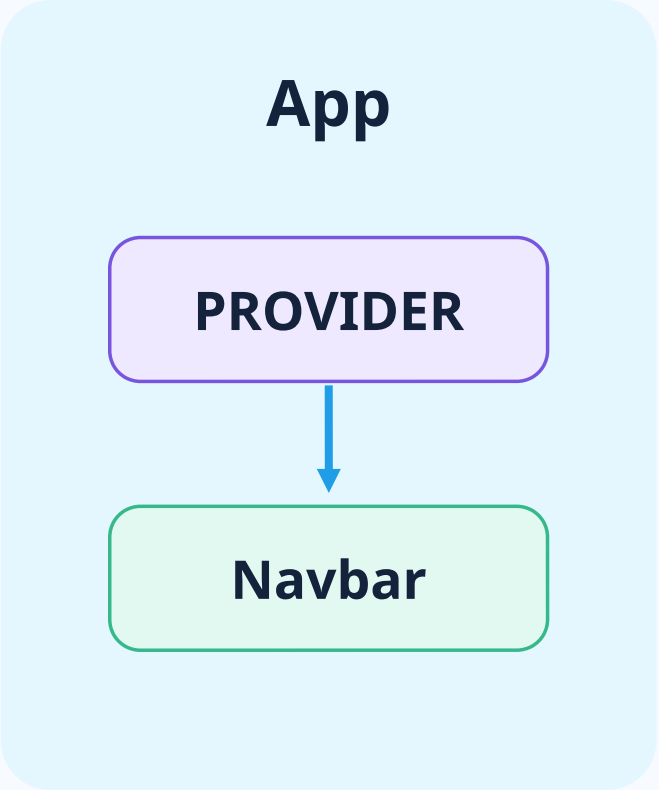
Used only when there is no matching provider above.

STEP 2 — Provide a value

Wrap the components that should receive the data.

```
import { ThemeContext } from './ThemeContext';

function App() {
  return (
    <ThemeContext value="dark">
      <Navbar />
    </ThemeContext>
  );
}
```



React 19+ supports this shorter provider syntax.

STEP 3 — Read with useContext

Only the component that needs the data reads it.

```
import { useContext } from 'react';
import { ThemeContext } from './ThemeContext';

function Button() {
  const theme = useContext(ThemeContext);
  return <button>{theme}</button>;
}
```

Button displays: dark

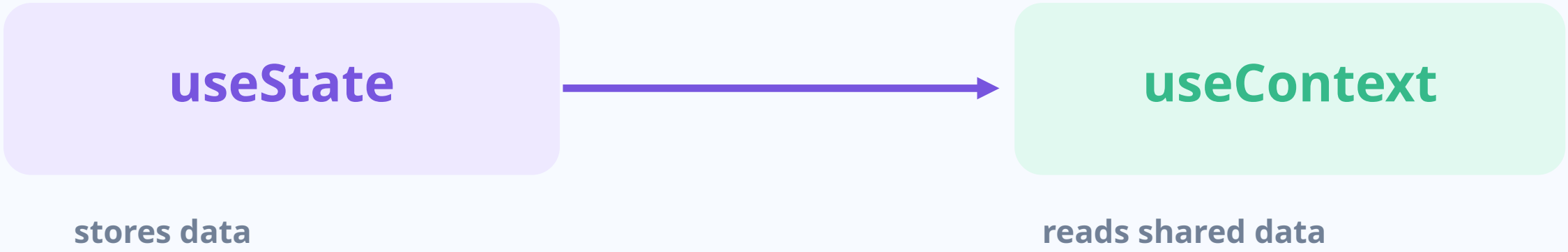
useContext finds the CLOSEST matching provider ABOVE.

Make it interactive!

Context shares state. useState actually stores and updates it.

```
const [theme, setTheme] = useState('light');

<ThemeContext value={{ theme, setTheme }}>
  <ThemeButton />
</ThemeContext>
```



Mini project: Theme Switcher

Click the button. The theme changes for every component using that context.

```
function ThemeButton() {
  const { theme, setTheme } =
    useContext(ThemeContext);

  return (
    <button onClick={() =>
      setTheme(theme === 'light' ? 'dark' : 'light')
    }>Switch theme</button>
  );
}
```

Challenge: change the page background too!

When should we use Context?

Context is useful, but you do NOT need it everywhere.

GOOD IDEAS

- App theme (light / dark)
- Language preference
- Signed-in user
- Many faraway consumers

USE PROPS INSTEAD

- One parent → one child
- A small, local value
- Simple reusable components
- When composition is enough

Two rules to remember

Avoid the most common beginner mistakes.

RULE 1

A consumer must be INSIDE the provider, not above it.

Hierarchy matters!

RULE 2

The CLOSEST provider wins when providers are nested.

Nearest value wins!

Bonus: useContext is a Hook — call it at the top level of your component.

YOUR MISSION: Build a Theme Control Center

Start with props → notice the problem → replace them with Context.

1. 1. Build App → Navbar → Menu → ThemeButton.
2. 2. Store theme with useState in App.
3. 3. Pass theme using props first.
4. 4. Refactor to createContext + Provider + useContext.
5. 5. Add a second reader: ThemeLabel.
6. **6. BONUS: make a nested provider with a different theme.**

EXPLAIN IN YOUR OWN WORDS: What problem did Context solve?

Remember the magic formula

Your cheat sheet + official documentation

CREATE → PROVIDE → READ

`createContext()` | `<Context value={...}>` | `useContext(Context)`

Context shares values. It does not replace state.

Official docs: react.dev/learn/passing-data-deeply-with-context